

Processamento Analítico em Larga Escala para Detecção de Fraudes: Uma Abordagem Baseada em Arquitetura Medallion e Apache Spark

Rafael Luciano Lima da Silva¹, Diêgo de Araujo Correia¹

¹Instituto de Computação – Universidade Federal de Alagoas (UFAL)

{rlls, dac}@ic.ufal.br

Resumo. A detecção de fraude em pagamentos combina uma classe positiva rara, custos assimétricos de erro e necessidade de resposta rápida. Este trabalho apresenta e avalia uma arquitetura lakehouse para o problema, integrando Apache Kafka, Spark Streaming e MLlib, HDFS, Iceberg, Hive Metastore e Airflow. O experimento usa as 284.807 transações do conjunto de transações de cartões de crédito MLG-ULB. Dois ensembles Random Forest–Gradient-Boosted Trees ponderados por classe são comparados em cinco execuções; a seleção usa somente PR-AUC de validação e o teste permanece isolado. O estudo produziu 284.807 scores, 562 alertas de fraude e resultados preditivos.

Abstract. Fraud detection in payments combines a rare positive class, asymmetric error costs, and the need for a rapid response. This work presents and evaluates a lakehouse architecture for the problem, integrating Apache Kafka, Spark Streaming, MLlib, HDFS, Iceberg, Hive Metastore, and Airflow. The experiment uses the 284,807 transactions from the MLG-ULB credit card transaction set. Two class-weighted Random Forest-Gradient-Boosted Trees ensembles are compared in five runs; selection uses only PR-AUC validation, and the test remains isolated. The study produced 284,807 scores, 562 fraud alerts, and predictive results.

1. Introdução

Pagamentos digitais tornaram a análise antifraude uma tarefa simultaneamente estatística e operacional. A decisão precisa ocorrer no intervalo crítico entre a chegada do evento e sua autorização, mas o resultado verdadeiro da transação normalmente só é conhecido posteriormente. Além disso, como uma fração ínfima dos registros representa fraude, uma acurácia elevada pode ser obtida por um classificador inútil que classifica todos os eventos como legítimos. A consequência prática da falha na predição é dupla e custosa: falsos negativos expõem a instituição a perdas financeiras diretas, enquanto falsos positivos interrompem clientes legítimos e aumentam expressivamente o custo de revisão humana.

Ao lado da complexidade do modelo preditivo, a infraestrutura de dados precisa preservar rigorosamente o caminho da informação. Uma solução que recebe eventos em *streaming*, realiza treinamentos em lote e depois produz análises isoladas tende a acumular sistemas desconexos, cópias redundantes de dados e regras difíceis de auditar. O conceito de data *lakehouse* procura solucionar esse gargalo ao combinar o armazenamento

aberto e escalável de um *data lake* com o controle de metadados, a confiabilidade de tabelas ACID e o desempenho analítico tradicionalmente associados ao *data warehouse* [Armbrust et al. 2021]. Essa convergência é altamente adequada ao cenário de fraudes financeiras: os mesmos dados podem sustentar a ingestão, a preparação, o treinamento de modelos, a inferência, os alertas e as consultas retrospectivas de negócio.

Neste contexto, este artigo apresenta uma implementação reprodutível de uma arquitetura *lakehouse* para detecção de fraude. A contribuição principal deste trabalho é o estabelecimento de uma *pipeline* experimental completa, escalável e auditável utilizando exclusivamente ferramentas de código aberto. O trabalho: (i) organiza os eventos em camadas consistentes (Bronze, Silver e Gold) sobre Apache Iceberg e HDFS; (ii) realiza a inferência analítica após a camada Silver e antes da Gold, preservando a ordem operacional estrita; (iii) define um protocolo de avaliação temporal isolado, mitigando o risco de vazamento de dados; e (iv) reporta métricas de classificação, escores, alertas operacionais e indicadores de qualidade.

2. Trabalhos Relacionados

A literatura sobre detecção de fraudes financeiras e processamento de dados em larga escala evoluiu de estudos focados puramente em algoritmos isolados para propostas de arquiteturas. O presente trabalho situa-se na interseção entre a modelagem preditiva com restrições temporais e a engenharia de dados.

No contexto da modelagem e avaliação de fraudes, [Ngai et al. 2011] realizaram uma revisão abrangente sobre a aplicação de técnicas de mineração de dados, destacando a complexidade estatística do domínio financeiro. Posteriormente, a literatura passou a enfatizar que o risco de pagamentos não deve ser tratado como um problema de classificação binária estática. Trabalhos seminais como os de [Dal Pozzolo et al. 2015, Dal Pozzolo et al. 2018] estabeleceram que a modelagem deve obrigatoriamente considerar o desbalanceamento extremo de classes, os custos assimétricos de decisão e a evolução temporal do comportamento transacional. Em particular, [Dal Pozzolo et al. 2018] recomendam fortemente a distinção entre o período de treinamento e um período futuro e estritamente cronológico de avaliação. O presente artigo incorpora essas diretrizes ao declarar métricas focadas na classe minoritária (PR-AUC) e implementar um rigoroso protocolo temporal sem vazamento de dados.

Oliveira e [Oliveira and Iyer 2023] e [Allam 2024] exploraram os benefícios de arquiteturas estruturadas em múltiplas camadas para o *lakehouse* e o papel da orquestração para garantir rastreabilidade e governança em ecossistemas financeiros.

Mais recentemente, publicações têm abordado a aplicação direta de *lakehouses* para o domínio específico de risco e pagamentos. Estudos conduzidos por [Adlermann 2024], [Battapothu 2025] e [Carrington 2025] reforçam a utilidade e a viabilidade de combinar o processamento de eventos em *streaming* com análises profundas em lote dentro do mesmo ecossistema para detecção de fraudes. Contudo, essas literaturas frequentemente descrevem arquiteturas em um nível puramente conceitual ou dependem pesadamente de ecossistemas e infraestruturas proprietárias em nuvem.

A principal lacuna que este artigo visa preencher em relação aos trabalhos correlatos é a aplicação ponta a ponta dessa teoria. O trabalho integra as rigorosas recomendações de avaliação temporal de [Dal Pozzolo et al. 2018] diretamente em um motor de

fluxo estruturado dentro de uma arquitetura *lakehouse*, utilizando um ambiente totalmente local e de código aberto (Apache Spark, Kafka e Iceberg), demonstrando a execução prática dos conceitos de integração e reprodutibilidade.

3. Descrição

3.1. Detecção de fraude e avaliação desbalanceada

Em detecção de fraude, a classe positiva é rara e os erros não têm o mesmo significado. Seja TP o número de fraudes corretamente sinalizadas, FP os alertas incorretos, TN as transações legítimas aceitas e FN as fraudes não sinalizadas. A precisão é $TP/(TP + FP)$, o recall é $TP/(TP + FN)$, enquanto o F1 harmoniza os dois critérios. Essas medidas devem ser lidas em conjunto: aumentar a sensibilidade reduzindo o limiar pode elevar fortemente os falsos positivos; elevar demasiadamente o limiar pode reduzir a fila de revisão, mas deixar fraudes passar.

A curva ROC e seu AUC continuam úteis para resumir ordenação de scores, mas podem parecer excessivamente favoráveis quando negativos dominam a população. A curva precisão–recall (PR) concentra a análise na classe de interesse e é recomendada para conjuntos desbalanceados [Davis and Goadrich 2006, Saito and Rehmsmeier 2015]. Por isso, este estudo declara PR-AUC como métrica primária para escolher hiperparâmetros. ROC-AUC, precisão, recall, F1 e a matriz de confusão são reportados como medidas complementares. A acurácia não é utilizada para seleção.

Outra decisão fundamental é a separação dos dados. Validação cruzada estratificada é comum em problemas estáticos, mas pode subestimar o erro quando há dependência temporal. Estudos de fraude recomendam distinguir o período de treinamento do período futuro de avaliação e tornar explícita a estratégia de atualização [Dal Pozzolo et al. 2018]. O protocolo deste trabalho usa três blocos cronológicos e não permite que o teste influencie a escolha de algoritmo, limiar ou pesos.

3.2. Lakehouse e tabelas transacionais abertas

O *lakehouse* emprega armazenamento de arquivos aberto, mas adiciona um catálogo e operações de tabela para oferecer consistência a *workloads* de engenharia e análise em alto volume de dados. O Apache Iceberg realiza esse papel no experimento: cada camada é uma tabela catalogada pelo Hive Metastore e mantida no HDFS. O particionamento físico dos dados no HDFS permite que consultas analíticas filtrem diretórios de forma eficiente, estratégia fundamental para a escalabilidade de leitura. O catálogo fornece um nome estável para o Spark, enquanto *snapshots* e metadados garantem a evolução e linhagem.

As camadas seguem a convenção *medallion*. Bronze retém o evento bruto, tópico, offset e *timestamp* de ingestão. Silver padroniza tipos, remove registros inválidos e disponibiliza as *features*. Gold recebe *scores*, alertas, KPIs e métricas de qualidade. Essa separação estrutural torna visível a origem de cada resultado e permite reprocessar a Silver ou a Gold sem republicar o arquivo de origem, sendo compatível com arquiteturas *multi-tier* discutidas para domínios financeiros regulados [Oliveira and Iyer 2023, Allam 2024].

3.3. Processamento distribuído, paralelismo e orquestração

O Apache Spark atua como motor unificado para processamento em memória, *streaming* estruturado, SQL e aprendizado de máquina [Zaharia et al. 2016]. O desempenho e a

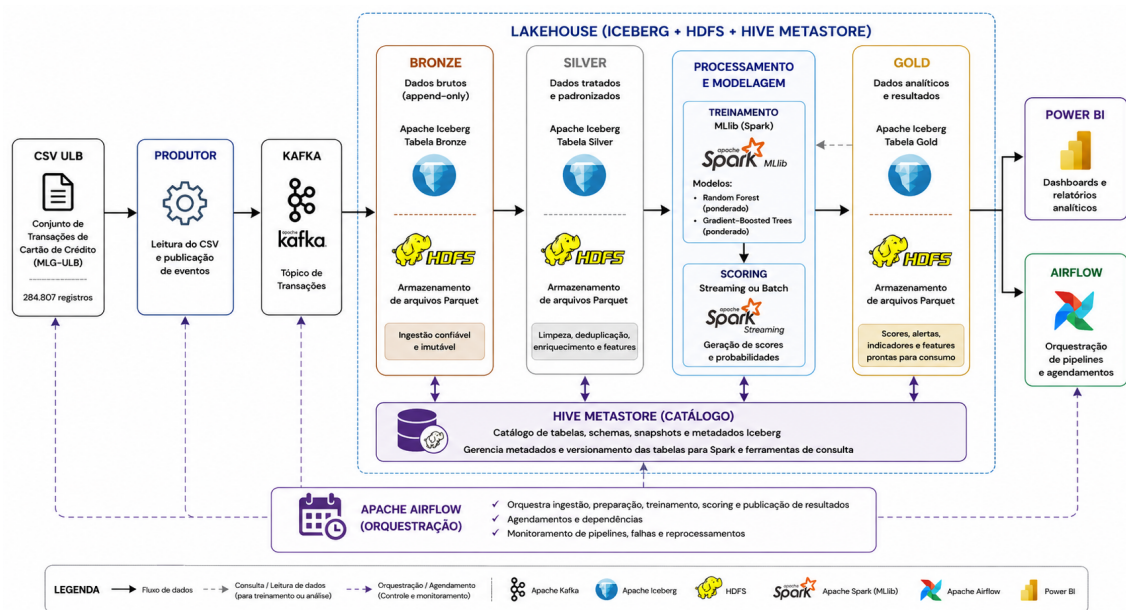


Figura 1. Visão geral da arquitetura experimental implementada

escalabilidade do motor derivam de sua capacidade de processamento distribuído: os *DataFrames* são divididos em partições lógicas e processados em paralelo por múltiplas tarefas (*tasks*) alocadas nos nós executores (*workers*) do *cluster*. No projeto, o mesmo ecossistema executa a limpeza *batch*, o treinamento MLlib, o *streaming* e as agregações, reduzindo a troca de formatos e o tráfego de rede.

O Apache Kafka é a ferramenta principal de ingestão e transmissão de eventos. Concebido como um sistema distribuído para alto *throughput* [Kreps et al. 2011], sua escalabilidade baseia-se no particionamento físico dos tópicos, permitindo leitura e escrita concorrentes. Ao acoplar as partições do Kafka com as do Spark via *Structured Streaming*, o processamento em *micro-batches* garante um fluxo resiliente.

Por fim, o Apache Airflow assume o papel de orquestrador. Suas DAGs disparam as rotinas de inicialização, processamento Silver, treinamento e qualidade. Ele não controla o laço contínuo de consumo do Kafka, pois os *streams* pertencem somente ao Spark.

4. Metodologia

4.1. Visão geral da arquitetura

A arquitetura foi implementada inteiramente em contêineres Docker Compose. Os serviços são Kafka e ZooKeeper, HDFS NameNode/DataNode, Hive Metastore, Spark Master e Worker, Airflow e Kafka UI. O armazenamento principal é HDFS; Iceberg usa Hive Catalog no endereço do metastore. As versões são fixadas no repositório: Spark 3.5.1, Iceberg 1.5.2, Kafka Confluent 7.6.1, Hadoop 3.2.1, Hive Metastore 3.1.3 e Airflow 2.9.2. A Figura 1 apresenta a organização dos componentes e o fluxo de dados entre as camadas da arquitetura experimental.

O desenho atende a dois caminhos. O caminho online recebe e persiste eventos, enquanto o caminho analítico produz modelos, analisa resultados e prepara indicadores.

O score é calculado depois da Silver, pois somente ali as features têm schema e tipos controlados; ele ocorre antes da Gold, pois Gold armazena somente o resultado da decisão. Essa ordem também permite que alertas e scores sejam publicados no Kafka sem aguardar agregações periódicas.

4.2. Ingestão e camada Bronze

O arquivo do dataset é validado antes da publicação. O produtor verifica a presença dos campos `Time`, `V1` a `V28`, `Amount` e `Class`; cada linha recebe um identificador e é enviada a Kafka. Na execução, foram usados lotes de 5.000 mensagens. O produtor publicou 284.807 eventos em 164,6 s, distribuídos em três partições do tópico. O checkpoint Bronze confirmou os offsets finais 95.515, 95.267 e 94.025, cuja soma é o número total de eventos.

O job `kafka_to_bronze.py` lê JSON, preserva o offset Kafka e grava em `local.bronze.transactions_raw`. A combinação de `transaction_id` e `offset` cria `raw_event_id`. O checkpoint fica no HDFS; assim, uma reinicialização do stream continua a partir do ponto processado. Essa etapa foi mantida mesmo para o conjunto histórico porque o objetivo experimental inclui validar a rota de ingestão que seria usada para dados em produção.

4.3. Silver: qualidade, schema e features

Silver é gerada pelo job `bronze_to_silver.py`. Todas as variáveis do dataset são convertidas explicitamente para `double`; a classe é disponibilizada como `label`; valores nulos das features são preenchidos com zero e labels fora de $\{0,1\}$ são excluídos. A tabela `local.silver.transactions_clean` preserva metadados técnicos e `local.silver.transactions_features` inclui um vetor de features para consumo analítico.

O desbalanceamento é tratado por pesos de classe no treinamento. Para cada classe c , o peso empregado é $w_c = N/(2N_c)$, em que N é o número de registros de treinamento e N_c o número de elementos da classe. Os pesos são calculados exclusivamente no bloco de treino em cada etapa de ajuste, não no conjunto completo. Essa estratégia preserva o conjunto original e evita introduzir sintetização externa, alinhando-se a uma comparação controlada com algoritmos Spark MLlib.

4.4. Protocolo experimental e prevenção de vazamento

O protocolo foi registrado em `configs/app/experiment.yaml`. A variável `Time` determina dois pontos de corte aproximados por quantil: 120.390 s e 145.229 s. O primeiro bloco contém 170.229 transações (60%), o segundo 56.732 (20%) e o bloco de teste 56.765 (20%). Os tamanhos diferem levemente das proporções nominais porque existem eventos com o mesmo instante; a regra de fronteira preserva o ordenamento. Os blocos contém, respectivamente, 323, 95 e 74 fraudes.

O experimento adota cinco sementes: 42, 52, 62, 72 e 82. Não existe um único número universal de repetições que seja "gold standard" para fraude; a literatura registra que a avaliação deve refletir a dependência temporal e a variabilidade do método [Dal Pozzolo et al. 2018, Hand 2009]. Cinco repetições são usadas para assegurar a estabilidade dos resultados em algoritmos estocásticos em ambiente local.

Dois candidatos foram definidos antes da execução. O candidato *compact* usa Random Forest com 80 árvores e profundidade máxima 8, mais GBT com 50 iterações e profundidade 3. O candidato *extended* usa 120 árvores/profundidade 10 e GBT de 60 iterações/profundidade 4. A grade é propositalmente pequena, pois árvores profundas e muitas iterações aumentariam o custo e risco de ajuste, enquanto duas capacidades permitem observar se maior complexidade melhora a ordenação PR no período de validação. Cada candidato é treinado cinco vezes sobre o mesmo bloco de treino.

Para cada ajuste, o score do ensemble é a média simples das probabilidades de fraude de Random Forest e GBT:

$$s(x) = \frac{p_{RF}(y = 1|x) + p_{GBT}(y = 1|x)}{2}. \quad (1)$$

O limiar é buscado na grade 0,05, 0,10, ..., 0,95. A política seleciona o maior recall com precisão mínima de 0,70; se nenhum limiar satisfizer o piso, usa o maior F1. A decisão de configuração usa a média de PR-AUC na validação, com F1 como desempate. Apenas após a escolha são carregados os cinco modelos vencedores e calculadas as métricas no teste. Por fim, um modelo de implantação é ajustado sobre treino mais validação; seu limiar e a mediana dos cinco limiares de validação. Esse último modelo é aplicado na Gold, mas suas métricas retrospectivas não substituem o teste isolado.

4.5. Treinamento, registro e inferência

O treinamento usa exclusivamente `RandomForestClassifier` e `GBTCClassifier` do Spark MLlib, ambos com `weightCol`. Cada modelo de validação é salvo no HDFS em um caminho que contém identificador do experimento, candidato e semente. As tabelas `local.gold.experiment_metrics` e `local.gold.experiment_summary` registram `seed`, `split`, limiar, matriz de confusão, PR-AUC, ROC-AUC, média e desvio-padrão. O arquivo `local.models/latest_metadata.json` registra versão, colunas, pesos, pontos de corte e caminhos dos modelos.

O job `score_silver_batch.py` aplica o modelo à toda à Silver. O job de streaming existente aplica a mesma lógica em micro-batches para novos registros. Em ambos os casos, `fraud_score`, probabilidades individuais, limiar, versão do modelo e instante de score são escritos em `local.gold.fraud_scores`; previsões positivas também entram em `local.gold.fraud_alerts`. A rotina Gold gera KPIs horários e diários e exporta CSV para Power BI. A rotina de qualidade mede contagem, nulidade, distribuições e outros indicadores, gravando 151 métricas na tabela Gold.

4.6. Ambiente e verificação

O experimento foi executado no ambiente local Docker do projeto, com um Spark Worker configurado com 2 cores e 3 GB de memória. A execução completa incluiu reinicialização dos volumes para evitar mistura com a amostra exploratória anterior, inicialização Iceberg, criação dos tópicos, ingestão, Bronze, Silver, experimento, scoring batch, Gold e qualidade. Os testes unitários do repositório foram executados no container Spark e obtiveram 12 testes aprovados.

Os tempos observados foram 164,6 s para publicação Kafka, 66,7 s para Bronze-Silver, 1.224,8 s para o experimento completo de modelos, 74,0 s para scoring batch, 44,7

s para agregações Gold e 68,2 s para qualidade. Esses tempos não são benchmark de escala, pois incluem um único worker e downloads iniciais de dependências no primeiro job. Eles documentam, contudo, que a pipeline foi executada de ponta a ponta e oferecem uma base para comparação de futuras configurações.

5. Experimento e resultados obtidos

5.1. Dataset ULB e integridade da execução

O conjunto Credit Card Fraud Detection foi disponibilizado pelo Machine Learning Group da Université Libre de Bruxelles em parceria com Worldline [Worldline and Machine Learning Group, Université Libre de Bruxelles 2013]. Ele contém transações de cartões europeus realizadas durante dois dias de setembro de 2013. Das 284.807 instâncias, 492 são fraudulentas, o que corresponde a aproximadamente 0,1727%. Por razões de confidencialidade, as componentes V1–V28 resultam de transformação PCA; Time representa segundos desde a primeira transação e Amount é o valor monetário. A label Class é usada apenas para treinamento e avaliação offline; em uma implantação real, o label chegaria com atraso.

A Tabela 1 confirma a integridade básica do fluxo. Bronze, Silver e Gold de scores preservaram as 284.807 transações. Isso mostra que o produtor, as três partições Kafka, o stream Iceberg e o job Silver não perderam ou duplicaram registros no experimento reinicializado. A tabela de alertas contém apenas a parcela classificada como suspeita; qualidade registrou 151 medidas auditáveis.

Tabela 1. Contagens materializadas por camada.

Artefato	Registros
Bronze transactions_raw	284.807
Silver transactions_clean	284.807
Gold fraud_scores	284.807
Gold fraud_alerts	562
Métricas de qualidade	151

5.2. Seleção por validação

A Tabela 2 mostra as médias das cinco sementes. O candidato *compact* foi escolhido porque sua PR-AUC média na validação foi 0,656087, ligeiramente superior a 0,649046 do *extended*. A diferença é pequena e os desvios-padrão se sobrepõem; portanto, o resultado não deve ser lido como evidência de superioridade ampla da configuração menor. Ele mostra, no entanto, que aumentar árvores, profundidade e iterações não trouxe ganho consistente no critério previamente declarado. O *extended* obteve precisão e F1 médias superiores nesse bloco, mas a regra de seleção manteve PR-AUC como objetivo principal para evitar trocar o critério após ver os resultados.

O limiar 0,70 foi selecionado em todas as cinco execuções do candidato compacto. Isso é coerente com a política de precisão mínima 0,70: no período de validação, o modelo pode elevar recall até o ponto em que a fila de alertas ainda preserva qualidade. O limiar do modelo de implantação foi a mediana desses valores, também 0,70. A estabilidade

Tabela 2. Validação temporal: média $\pm$ desvio-padrão em cinco sementes. O vencedor foi definido por PR-AUC.

Candidato	PR-AUC	ROC-AUC	Precision	Recall	F1	Limiar
Compact	0,66 $\pm$ 0,03	0,97 $\pm$ 0,00	0,80 $\pm$ 0,05	0,74 $\pm$ 0,02	0,77 $\pm$ 0,03	0,70 $\pm$ 0,00
Extended	0,65 $\pm$ 0,05	0,96 $\pm$ 0,01	0,86 $\pm$ 0,06	0,72 $\pm$ 0,03	0,78 $\pm$ 0,03	0,58 $\pm$ 0,03

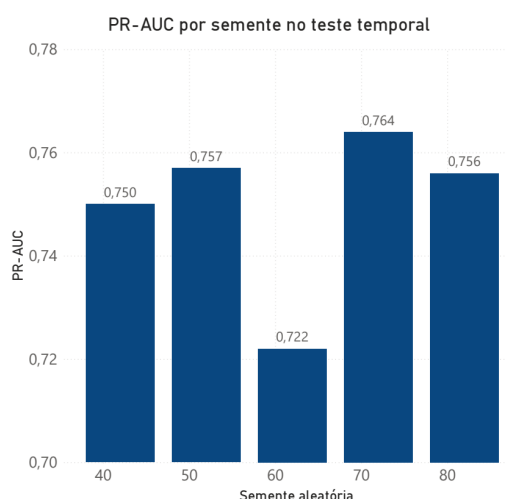


Figura 2. PR-AUC do candidato compacto nas cinco sementes do teste temporal.

do limiar não elimina a necessidade de recalibração futura, pois o dataset é histórico e não representa deriva de conceito em longo prazo. A variação da PR-AUC entre as cinco sementes no bloco temporal de teste é apresentada na Figura 2.

5.3. Teste cronológico isolado

O bloco de teste contém 56.765 transações, das quais 74 são fraudes. A Tabela 3 apresenta cada repetição do candidato selecionado. A média de PR-AUC foi 0,749825 com desvio-padrão 0,016599; ROC-AUC foi 0,978700 com desvio-padrão 0,004384. Precision média de 0,861579 significa que a maior parte dos alertas do teste correspondeu a fraude, enquanto recall médio de 0,735135 indica que cerca de três quartos das 74 fraudes foram sinalizadas no limiar escolhido. O F1 médio foi 0,793038.

Os resultados mostram baixa variação em recall e F1, e variação moderada em PR-AUC. Essa variação é esperada porque há apenas 74 positivos no teste e porque Random Forest e GBT possuem componentes estocásticos. O ponto mais baixo de PR-AUC ocorreu na semente 62, embora sua precisão tenha sido a maior; novamente, uma única métrica não descreve todos os trade-offs. A avaliação por repetição é mais informativa que reportar somente o melhor run, prática que poderia superestimar a capacidade do método.

O teste também evidência um ponto de interpretação: PR-AUC no teste foi maior que na validação. Isso não deve ser tratado como melhoria causada pelo teste, mas como diferença de dificuldade entre janelas temporais e tamanho pequeno da classe positiva. Como o teste não participou da escolha de configuração, ele continua válido para a estimativa deste protocolo. A diferença reforça a importância de executar reavaliações tem-

Tabela 3. Resultados por semente no bloco temporal de teste.

Run	Seed	Precision	Recall	F1	PR-AUC	ROC-AUC	TP	FP	TN	FN
1	42	0,86	0,76	0,81	0,75	0,97	56	9	56.682	18
2	52	0,84	0,73	0,78	0,76	0,98	54	10	56.681	20
3	62	0,90	0,72	0,80	0,72	0,97	53	6	56.685	21
4	72	0,87	0,73	0,79	0,76	0,98	54	8	56.683	20
5	82	0,83	0,74	0,79	0,76	0,98	55	11	56.680	19
Média	–	0,86	0,74	0,79	0,75	0,98	–	–	–	–
DP	–	0,03	0,02	0,01	0,02	0,00	–	–	–	–

porais sucessivas em cenários reais.

5.4. Scores e alertas do modelo de implantação

Depois de concluída a avaliação, o modelo compacto de implantação foi treinado com treino e validação e aplicado retrospectivamente a toda a Silver. Esta etapa não é uma nova medida de generalização, porque inclui registros vistos no treino; seu objetivo é demonstrar o produto operacional da pipeline. Foram persistidos 284.807 scores e 562 alertas a limiar 0,70. Dos alertas, 435 eram fraudes reais e 127 eram falsos positivos; 57 fraudes não foram alertadas. A precisão retrospectiva foi $435/562 = 0,7731$ e o recall retrospectivo foi $435/492 = 0,8841$.

O recall operacional maior que o recall de teste é esperado porque o modelo de implantação usa 80% dos dados para treinamento e é aplicado também sobre esse período. Por essa razão, a Tabela 3, e não estes números retrospectivos, é a referência científica para desempenho preditivo. Os scores Gold, contudo, são valiosos para auditoria, priorização e BI: cada linha preserva probabilidades de RF e GBT, score final, predição, limiar, versão do modelo e timestamp de scoring. A distribuição dos scores produzidos pelo modelo de implantação e a posição do limiar operacional podem ser observadas na Figura 3.

Tabela 4. Materialização operacional sobre toda a Silver. não e avaliação independente.

Medida	Valor
Scores produzidos	284.807
Fraudes rotuladas	492
Alertas	562
Verdadeiros positivos	435
Falsos positivos	127
Falsos negativos	57
Precision retrospectiva	0,7731
Recall retrospectivo	0,8841

5.5. Desempenho da pipeline e qualidade

A execução confirma que a arquitetura suporta o fluxo completo no ambiente local. A publicação Kafka levou 164,6 s e o experimento, que inclui dez ajustes de validação mais

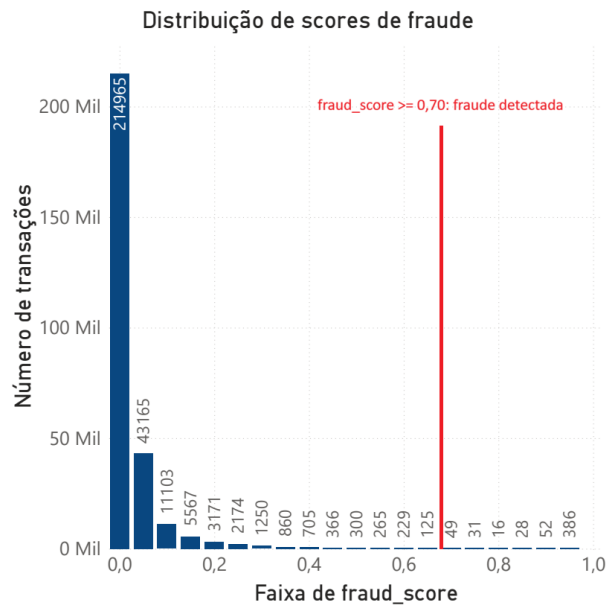


Figura 3. Distribuição de `fraud_score` para as transações avaliadas. A linha vermelha indica o limiar de 0,70 para gerar alertas de fraude.

o modelo de implantação, levou 1.224,8 s (aproximadamente 20,4 min). O *scoring batch* final levou 74,0 s. Esses números são observações do ambiente de teste, e não afirmações de *throughput* universal: há apenas um *worker*, dois *cores* e 3 GB de memória. Ainda assim, o fato de todo o conjunto atravessar Kafka, Iceberg e Spark sem perda fornece evidência de integração funcional.

Embora o *dataset* ULB possua 284.807 registros, um volume relativamente pequeno para o conceito de volume em *Big Data*, a arquitetura proposta foi desenhada para processamento em larga escala. O uso de *DataFrames* do Spark em conjunto com o armazenamento colunar (Parquet) via Iceberg garante que essa mesma *pipeline* processaria terabytes de dados sem necessidade de alteração no código-fonte. A escalabilidade para cenários massivos ocorreria exclusivamente de forma horizontal, mediante a adição de novos nós ao *cluster* Spark (aumentando o paralelismo de *tasks*) e ao HDFS (aumentando a capacidade de I/O e armazenamento particionado).

As 151 métricas de qualidade incluem contagens, completude e distribuições por tabela. A igualdade das contagens entre Bronze, Silver e Gold *scores* é uma verificação simples e forte para esta execução. Em produção, esse conjunto deveria ser complementado por alertas de deriva de distribuição, atraso de *label*, mudança de precisão estimada e SLA por *micro-batch*. A estrutura Iceberg permite que essas verificações sejam escritas como tabelas, em vez de *logs* efêmeros.

6. Conclusão

Este trabalho desenvolveu e executou uma arquitetura lakehouse local para detecção de fraude em cartões. A pipeline integrou Kafka para eventos, Spark Structured Streaming para Bronze e inferência, HDFS e Iceberg para persistência de camadas, Hive Metastore para catálogo, Airflow para orquestração e Spark MLlib para treinamento. O experimento ULB percorreu todas as 284.807 transações, gerou 284.807 scores, 562 alertas, KPIs,

exportações CSV e 151 métricas de qualidade.

Do ponto de vista metodológico, a principal decisão foi substituir uma divisão aleatória simples por protocolo temporal 60/20/20, com validação separada, teste isolado e cinco sementes. O candidato compacto foi selecionado por PR-AUC de validação e obteve PR-AUC médio de 0,749825 e precisão média de 0,861579 no teste. Os valores mostram que o modelo identifica uma parcela relevante das fraudes mantendo uma fila de alertas relativamente precisa, mas também deixam claro que ainda há falsos negativos e que a escolha do limiar depende do custo real de revisão e perda.

Há limitações importantes. O ULB é histórico, tem features anonimizadas por PCA e representa somente dois dias; logo, não demonstra deriva de conceito ou comportamento de clientes em longo prazo. O experimento utiliza um único worker local e não mede escalabilidade horizontal, latência por evento ou disponibilidade sob falha. A grade de hiperparâmetros foi pequena, os modelos não incluem contexto de entidade e o ensemble por média não substitui uma análise de custo monetário.

Como trabalhos futuros, propõe-se avaliar janelas temporais deslizantes e pre-quential, incorporar custos por transação, comparar diferentes políticas de recalibração, medir throughput com múltiplos workers e adicionar monitoramento de deriva. O estudo pode ser reproduzido pelo código no GitHub em https://github.com/rafaellucian0/fraud_detection-big_data, que inclui Docker Compose, configurações, jobs Spark, DAGs Airflow, testes e instruções para posicionar o dataset ULB.

Referências

- Adlermann, J. F. (2024). A gra-enhanced cloud ai framework for petabyte-scale multi-tenant environments: Multivariate classification for credit card fraud detection and adaptive risk analytics. *International Journal of Engineering & Extended Technologies Research*, 6(6).
- Allam, K. (2024). Cloud-based data hubs and sql pipelines for real-time financial analytics. *International Journal of Emerging Trends in Computer Science and Information Technology*, 5(4):94–104.
- Armbrust, M., Ghodsi, A., Xin, R., and Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *Proceedings of CIDR 2021*.
- Battapothu, V. (2025). Lakehouse-powered payment risk at scale. Manuscrito tecnico independente.
- Carrington, N. L. (2025). Ai-enabled cloud lakehouse for large-scale data warehousing: Sap integration for secure analytics and tableau-driven reporting. *International Journal of Research and Applied Innovations*, 8(5).
- Dal Pozzolo, A., Caelen, O., Johnson, R. A., and Bontempi, G. (2018). Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8):3784–3797.

- Dal Pozzolo, A., Caelen, O., Le Borgne, Y.-A., Waterschoot, S., and Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. In *IEEE Symposium Series on Computational Intelligence*, pages 159–166.
- Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and roc curves. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 233–240.
- Hand, D. J. (2009). Measuring classifier performance: A coherent alternative to the area under the roc curve. *Machine Learning*, 77:103–123.
- Kreps, J., Narkhede, N., and Rao, J. (2011). Kafka: A distributed messaging system for log processing. In *Proceedings of NetDB*, pages 1–7.
- Ngai, E. W. T., Hu, Y., Wong, Y. H., Chen, Y., and Sun, X. (2011). The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision Support Systems*, 50(3):559–569.
- Oliveira, M. and Iyer, R. (2023). Migrating bfsi data workloads to cloud-native environments: A case study on multi-tier data lakehouse architectures with aws redshift, athena, and intelligent orchestration for compliance. *Journal of Engineering, Mechanics and Modern Architecture*, 2(11):50–61.
- Saito, T. and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432.
- Worldline and Machine Learning Group, Universite Libre de Bruxelles (2013). Credit card fraud detection dataset. Kaggle Dataset: [mlg-ulb/creditcardfraud](https://www.kaggle.com/mlg-ulb/creditcardfraud).
- Zaharia, M., Xin, R. S., Wendell, P., et al. (2016). Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65.